

Restaurant Management API

Documentação completa do sistema

Tech Challenge - Fase 01 - P?s-Tech em Arquitetura e Desenvolvimento Java

Reposit?rio GitHub: <https://github.com/Andrefelix2810/restaurant-management-api>

Documento gerado a partir do README.md atualizado do projeto, com os requisitos do enunciado, arquitetura, endpoints, execu??o, Docker Compose, persist?ncia JDBC, collection Postman e crit?rios de valida??o.

Restaurant Management API

Projeto desenvolvido para o Tech Challenge — Fase 01 da Pós-Tech em Arquitetura e Desenvolvimento Java. A aplicação consiste em um backend REST para gerenciamento de usuários de um sistema compartilhado de restaurantes, contemplando cadastro, consulta, atualização, exclusão, troca de senha e validação de login, com persistência em banco relacional PostgreSQL e execução via Docker Compose.

Repositório GitHub: <https://github.com/Andrefelix2810/restaurant-management-api>

Documento base: Postech - ADJ - Fase 1 - Tech Challenge.pdf

Sumário

1. [Contexto Acadêmico](#contexto-acadêmico)
2. [Repositório de Código](#repositório-de-código)
3. [Objetivo do Projeto](#objetivo-do-projeto)
4. [Escopo da Fase 01](#escopo-da-fase-01)
5. [Requisitos do Enunciado e Atendimento](#requisitos-do-enunciado-e-atendimento)
6. [Tecnologias Utilizadas](#tecnologias-utilizadas)
7. [Arquitetura da Solução](#arquitetura-da-solução)
8. [Qualidade do Código e Organização](#qualidade-do-código-e-organização)
9. [Modelo de Domínio](#modelo-de-domínio)
10. [Persistência JDBC e Modelo Relacional](#persistência-jdbc-e-modelo-relacional)
11. [Funcionalidades Implementadas](#funcionalidades-implementadas)
12. [Endpoints da API](#endpoints-da-api)
13. [Exemplos de Requisição](#exemplos-de-requisição)
14. [Tratamento de Erros](#tratamento-de-erros)
15. [Documentação Swagger / OpenAPI](#documentação-swagger--openapi)
16. [Collections Postman](#collections-postman)
17. [Configuração e Execução](#configuração-e-execução)
18. [Docker e Docker Compose](#docker-e-docker-compose)
19. [Build, Testes e Validação](#build-testes-e-validação)
20. [Estrutura do Projeto](#estrutura-do-projeto)
21. [Aderência aos Requisitos da Fase 01](#aderência-aos-requisitos-da-fase-01)
22. [Decisões Técnicas](#decisões-técnicas)
23. [Possíveis Evoluções Futuras](#possíveis-evoluções-futuras)
24. [Conclusão](#conclusão)

Contexto Acadêmico

O Tech Challenge — Fase 01 propõe a construção de um backend robusto para apoiar um sistema de gestão compartilhado por restaurantes.

O problema apresentado considera um grupo de restaurantes que, diante do alto custo de soluções individuais, decidiu construir uma plataforma comum para gerenciamento de suas operações. Dessa forma, os restaurantes passam a compartilhar uma solução tecnológica única, permitindo que os clientes escolham os estabelecimentos com base na qualidade da comida, atendimento e experiência oferecida, e não pela limitação técnica de seus sistemas de gestão.

Nesta primeira fase, o foco está concentrado no desenvolvimento do backend responsável pelo gerenciamento de usuários, permitindo que clientes e donos de restaurantes sejam cadastrados e administrados por meio de uma API REST.

Repositório de Código

Conforme solicitado nos entregáveis da Fase 01, o código-fonte da aplicação está disponível em um repositório público no GitHub:

```
[text]  
https://github.com/Andrefelix2810/restaurant-management-api
```

O repositório permite baixar o projeto, consultar a estrutura de implementação, validar os arquivos de configuração, importar a collection Postman e executar a aplicação localmente ou via Docker Compose.

Objetivo do Projeto

O objetivo principal deste projeto é disponibilizar uma API backend desenvolvida com Spring Boot, estruturada em camadas e preparada para execução em ambiente containerizado.

A aplicação permite realizar operações essenciais sobre usuários, incluindo:

- ? cadastro de clientes;
- ? cadastro de donos de restaurante;
- ? consulta de usuários cadastrados;
- ? consulta de usuário por identificador;
- ? atualização de dados cadastrais;
- ? exclusão de usuários;
- ? troca de senha;
- ? validação de login.

A solução utiliza PostgreSQL como banco de dados relacional e Docker Compose para orquestrar a aplicação Java e o serviço de banco de dados, facilitando a reprodução do ambiente em diferentes máquinas.

Escopo da Fase 01

O escopo desta fase contempla exclusivamente funcionalidades de backend relacionadas ao usuário. A API não possui interface gráfica, pois a proposta da fase concentra-se na construção da camada de serviços e exposição de endpoints REST.

Tipos de usuário suportados

Tipo	Descrição
CLIENT	Representa o cliente que poderá utilizar o sistema para consultar restaurantes, realizar pedidos e deixar avaliações em fases futuras.
RESTAURANT_OWNER	Representa o dono de restaurante responsável pela administração do estabelecimento em fases posteriores.

Dados cadastrais considerados

Campo	Tipo	Obrigatório	Observação
name	String	Sim	Nome completo do usuário.
email	String	Sim	Deve possuir formato válido e ser único.
login	String	Sim	Identificador de acesso do usuário e deve ser único.
password	String	Sim	Senha com no mínimo 6 caracteres.
lastModifiedDate	Date/Time	Gerado pelo sistema	Atualizado na criação e alteração do cadastro/senha.
address	Object	Sim	Endereço completo do usuário.

Requisitos do Enunciado e Atendimento

O enunciado da Fase 01 solicita um backend completo com Spring Boot, focado em gerenciamento de usuários, com execução em Docker Compose, banco relacional, documentação, collection de testes e repositório aberto. A tabela abaixo consolida como cada ponto foi atendido neste projeto.

Solicitação do PDF	Implementação neste projeto	Evidência
Backend completo e robusto em Spring Boot	API REST organizada em camadas com controllers, services, repository, DTOs, mapper e tratamento global de exceções.	src/main/java/com/restaurantssystem/restaurantmanagementapi
Foco em gerenciamento de usuários	Endpoints de cadastro, consulta, atualização, exclusão, troca de senha e validação de login.	UserController e AuthController
Dois tipos de usuário	Enum Role com os perfis CLIENT e RESTAURANT_OWNER.	src/main/java/.../enums/Role.java
Cadastro com nome, email, login, senha, data da última alteração e endereço	DTOs de entrada, objeto User, objeto Address e persistência na tabela users.	UserCreateRequest, User, Address, schema.sql
Alteração de dados do usuário	Endpoint PUT /users/{id} com validação de e-mail/login duplicado.	UserServiceImpl.update
Troca de senha	Endpoint PATCH /users/{id}/password, conferindo senha antiga e impedindo nova senha igual à atual.	UserServiceImpl.updatePassword
Validação de login	Endpoint POST /auth/login, validando login e senha.	AuthController e UserServiceImpl.login
Banco de dados relacional	PostgreSQL configurado via datasource Spring e Docker Compose.	application.properties e docker-compose.yml
Docker Compose	Orquestração de app e postgres, com healthcheck do banco e dependência da aplicação.	docker-compose.yml
Documentação do projeto	README completo com arquitetura, endpoints, exemplos, configuração, Docker, validação e decisões técnicas.	README.md
Collection para testes	Collection Postman disponível para importação e execução manual dos endpoints.	postman/Restaurant-API-Local.postman_collection.json
Repositório aberto	Código-fonte publicado no GitHub.	https://github.com/Andrefelix2810/restaurant-management-api

Tecnologias Utilizadas

Tecnologia	Finalidade
Java 17	Linguagem principal do projeto.
Spring Boot	Framework base para construção da aplicação.
Spring Web MVC	Exposição dos endpoints REST.
Spring JDBC	Persistência e acesso ao banco de dados com SQL explícito.
Jakarta Bean Validation	Validação dos dados de entrada.
PostgreSQL	Banco de dados relacional utilizado pela aplicação.
Maven	Gerenciamento de dependências e build do projeto.
Docker	Criação da imagem da aplicação.
Docker Compose	Orquestração da API e do banco PostgreSQL.
Lombok	Redução de código repetitivo em classes de domínio e DTOs.
Springdoc OpenAPI	Documentação interativa da API via Swagger UI.
Postman	Apoio para testes manuais dos endpoints.

Arquitetura da Solução

A aplicação foi organizada seguindo uma arquitetura em camadas, favorecendo separação de responsabilidades, manutenibilidade e evolução incremental.

```
[text]
Controller  -> Service  -> Repository  -> Database
    |           |           |
    DTO        Regras    JDBC
Validação    de negócio PostgreSQL
```

Responsabilidade das camadas

Camada	Responsabilidade
controller	Recebe as requisições HTTP, valida os contratos de entrada e retorna respostas adequadas.
dto.request	Define os objetos de entrada da API, isolando o contrato externo do modelo interno.
dto.response	Define os objetos de resposta devolvidos pela API.
entity	Representa as entidades persistidas no banco de dados.
repository	Realiza a comunicação com o banco por meio do Spring JDBC.
service	Define as operações de negócio disponíveis.
service.impl	Implementa as regras de negócio, validações e orquestração de persistência.
mapper	Converte entidades em DTOs e DTOs em entidades.
exception	Centraliza exceções e padroniza respostas de erro.
Config	Agrupar configurações auxiliares, como OpenAPI/Swagger.

Qualidade do Código e Organização

O projeto foi estruturado para manter separação clara entre entrada HTTP, regras de negócio, persistência e representação de dados. Essa organização atende ao fator de avaliação de qualidade do código indicado no PDF.

Padrões aplicados

Prática	Aplicação
Separação por camadas	Controllers recebem requisições; services concentram regras; repository executa SQL; DTOs representam contratos externos.
DTOs de entrada e saída	A API não expõe diretamente os objetos internos nem retorna senha nas respostas.
Validação declarativa	Bean Validation com @Valid, @NotBlank, @NotNull, @Email e @Size.
Tratamento centralizado de erro	GlobalExceptionHandler padroniza respostas para validações, regras de negócio, usuário inexistente e erros inesperados.
Persistência explícita	UserRepository usa Spring JDBC com SQL claro para consulta e manipulação dos dados.
Documentação interativa	Springdoc OpenAPI disponibiliza Swagger UI para leitura e teste dos contratos.
Configuração reprodutível	Docker Compose cria o banco PostgreSQL e sobe a aplicação com variáveis de ambiente padronizadas.

Fluxo principal de cadastro

```
[mermaid]
sequenceDiagram
    participant Client as Cliente HTTP
    participant Controller as UserController
    participant Service as UserServiceImpl
    participant Repository as UserRepository JDBC
    participant Database as PostgreSQL

    Client->>Controller: POST /users/register/client
    Controller->>Service: create(request, CLIENT)
    Service->>Repository: existsByEmail / existsByLogin
    Repository->>Database: SELECT COUNT
    Service->>Repository: save(user)
    Repository->>Database: INSERT INTO users
    Repository-->>Service: User com ID gerado
    Service-->>Controller: UserResponse
    Controller-->>Client: 201 Created
```

Modelo de Domínio

O domínio principal da Fase 01 é composto pela entidade User, que possui os dados cadastrais, credenciais de acesso, tipo de perfil e endereço.

```
[mermaid]
classDiagram
    class User {
        Long id
        String name
        String email
        String login
        String password
        LocalDateTime lastModifiedDate
        Role role
        Address address
    }
    class Address {
        String street
        String number
        String neighborhood
        String city
        String state
        String zipCode
        String complement
    }
    class Role {
        CLIENT
        RESTAURANT_OWNER
    }
    User --> Address
    User --> Role
```

Entidade User

Atributo	Descrição
id	Identificador único gerado automaticamente.
name	Nome do usuário.
email	E-mail único do usuário.
login	Login único utilizado para autenticação.
password	Senha do usuário.
lastModifiedDate	Data e hora da última alteração relevante.
role	Tipo do usuário: cliente ou dono de restaurante.
address	Endereço completo associado ao usuário.

Objeto Address

Atributo	Descrição
street	Rua ou avenida.
number	Número do endereço.
neighborhood	Bairro.
city	Cidade.
state	Estado.
zipCode	CEP.
complement	Complemento opcional.

Persistência JDBC e Modelo Relacional

O projeto utiliza Spring JDBC para acesso ao banco de dados. A camada repository trabalha com SQL explícito por meio de `NamedParameterJdbcTemplate`, evitando dependência de JPA/Hibernate e deixando as operações de persistência transparentes.

Tabela users

A estrutura da tabela é criada pelo arquivo `src/main/resources/schema.sql` durante a inicialização da aplicação, conforme a propriedade `spring.sql.init.mode`.

Coluna	Tipo	Restrição	Origem no domínio
id	BIGSERIAL	Chave primária	User.id
name	VARCHAR(255)	Opcional no banco, obrigatório na API	User.name
email	VARCHAR(255)	NOT NULL, UNIQUE	User.email
login	VARCHAR(255)	NOT NULL, UNIQUE	User.login
password	VARCHAR(255)	NOT NULL	User.password
last_modified_date	TIMESTAMP	NOT NULL	User.lastModifiedDate
role	VARCHAR(50)	NOT NULL	User.role
street	VARCHAR(255)	Endereço	Address.street
number	VARCHAR(50)	Endereço	Address.number

Coluna	Tipo	Restrição	Origem no domínio
neighborhood	VARCHAR(255)	Endereço	Address.neighborhood
city	VARCHAR(255)	Endereço	Address.city
state	VARCHAR(100)	Endereço	Address.state
zip_code	VARCHAR(30)	Endereço	Address.zipCode
complement	VARCHAR(255)	Endereço opcional	Address.complement

Operações implementadas no repository

Método	Operação SQL equivalente	Finalidade
save(User)	INSERT ou UPDATE	Criação e atualização de usuários.
findAll()	SELECT ... FROM users ORDER BY id	Listagem geral.
findById(Long)	SELECT ... WHERE id = :id	Busca por identificador.
findByEmail(String)	SELECT ... WHERE email = :email	Validação de unicidade de e-mail.
findByLogin(String)	SELECT ... WHERE login = :login	Login e validação de unicidade.
existsByEmail(String)	SELECT COUNT(1)	Verifica se e-mail já existe.
existsByLogin(String)	SELECT COUNT(1)	Verifica se login já existe.
delete(User)	DELETE FROM users WHERE id = :id	Exclusão de usuário.

Funcionalidades Implementadas

Gestão de usuários

- ? Registro de usuários com perfil de cliente.
- ? Registro de usuários com perfil de dono de restaurante.
- ? Listagem de todos os usuários.
- ? Busca de usuário por ID.
- ? Atualização de dados cadastrais.
- ? Exclusão de usuário.
- ? Atualização automática da data de última modificação.

Validação de dados

- ? Validação de campos obrigatórios com @NotBlank e @NotNull.
- ? Validação de formato de e-mail com @Email.
- ? Validação de tamanho mínimo de senha com @Size(min = 6).
- ? Validação de endereço obrigatório.
- ? Validação de unicidade de e-mail.
- ? Validação de unicidade de login.

Validação de login

- ? Validação de login por meio do endpoint `/auth/login`.
- ? Verificação de login e senha informados.
- ? Retorno dos dados do usuário autenticado em caso de sucesso.
- ? Retorno padronizado de erro para credenciais inválidas.

Troca de senha

- ? Verificação da senha antiga antes da alteração.
- ? Bloqueio de nova senha igual à senha atual.
- ? Atualização da data de última modificação após alteração.

Endpoints da API

Health check

Método	Endpoint	Descrição	Status esperado
GET	<code>/health</code>	Verifica se a API está em execução.	200 OK

Autenticação

Método	Endpoint	Descrição	Status esperado
POST	<code>/auth/login</code>	Valida login e senha do usuário.	200 OK

Usuários

Método	Endpoint	Descrição	Status esperado
POST	<code>/users/register/client</code>	Cadastra um usuário com perfil CLIENT.	201 Created
POST	<code>/users/register/owner</code>	Cadastra um usuário com perfil RESTAURANT_OWNER.	201 Created
GET	<code>/users</code>	Lista todos os usuários cadastrados.	200 OK
GET	<code>/users/{id}</code>	Busca um usuário pelo ID.	200 OK
PUT	<code>/users/{id}</code>	Atualiza os dados cadastrais de um usuário.	200 OK
PATCH	<code>/users/{id}/password</code>	Altera a senha de um usuário.	204 No Content
DELETE	<code>/users/{id}</code>	Remove um usuário pelo ID.	204 No Content

Exemplos de Requisição

1. Registrar cliente

```
[http]
POST /users/register/client
Content-Type: application/json

[json]
{
  "name": "Luis Ricardo",
  "email": "luis.ricardo@email.com",
  "login": "luisricardo",
  "password": "senha123",
  "address": {
    "street": "Avenida Paulista",
    "number": "1000",
    "neighborhood": "Bela Vista",
    "city": "São Paulo",
    "state": "SP",
    "zipCode": "01310100",
    "complement": "Bloco B, Apto 42"
  }
}
```

Resposta esperada

```
[json]
{
  "id": 1,
  "name": "Luis Ricardo",
  "email": "luis.ricardo@email.com",
  "login": "luisricardo",
  "lastModifiedDate": "2026-05-02T10:00:00",
  "role": "CLIENT",
  "address": {
    "street": "Avenida Paulista",
    "number": "1000",
    "neighborhood": "Bela Vista",
    "city": "São Paulo",
    "state": "SP",
    "zipCode": "01310100",
    "complement": "Bloco B, Apto 42"
  }
}
```

2. Registrar dono de restaurante

```
[http]
POST /users/register/owner
Content-Type: application/json

[json]
{
  "name": "Mariana Souza",
  "email": "mariana.souza@email.com",
  "login": "marianasouza",
  "password": "senha123",
  "address": {
    "street": "Rua Augusta",
    "number": "500",
    "neighborhood": "Consolação",
    "city": "São Paulo",
    "state": "SP",
    "zipCode": "01305000",
    "complement": "Sala 10"
  }
}
```

3. Realizar login

```
[http]
POST /auth/login
Content-Type: application/json
```

```
[json]
{
  "login": "luisricardo",
  "password": "senha123"
}
```

4. Atualizar usuário

```
[http]
PUT /users/1
Content-Type: application/json

[json]
{
  "name": "Luis Ricardo Almeida",
  "email": "luis.almeida@email.com",
  "login": "luisalmeida",
  "address": {
    "street": "Rua Vergueiro",
    "number": "2000",
    "neighborhood": "Vila Mariana",
    "city": "São Paulo",
    "state": "SP",
    "zipCode": "04101000",
    "complement": "Casa 2"
  }
}
```

5. Alterar senha

```
[http]
PATCH /users/1/password
Content-Type: application/json

[json]
{
  "oldPassword": "senha123",
  "newPassword": "novaSenha123"
}
```

6. Excluir usuário

```
[http]
DELETE /users/1
```

Resposta esperada:

```
[text]
204 No Content
```

Tratamento de Erros

O projeto possui tratamento global de exceções com `@RestControllerAdvice`, garantindo respostas padronizadas para erros de validação, regras de negócio e recursos inexistentes.

Formato padrão de erro

```
[json]
{
  "timestamp": "2026-05-02T10:30:00",
  "status": 400,
  "error": "Bad Request",
  "message": "Email already registered",
  "path": "/users/register/client"
}
```

Principais cenários tratados

Cenário	Exceção/Validação	Status
Usuário inexistente	UserNotFoundException	404 Not Found
E-mail em formato inválido	Bean Validation	400 Bad Request
Campo obrigatório ausente	Bean Validation	400 Bad Request
E-mail já cadastrado	BusinessException	400 Bad Request
Login já cadastrado	BusinessException	400 Bad Request
Senha antiga incorreta	BusinessException	400 Bad Request
Credenciais inválidas	BusinessException	400 Bad Request
Erro não previsto	Exception	500 Internal Server Error

Documentação Swagger / OpenAPI

A aplicação possui documentação interativa por meio do Springdoc OpenAPI.

Após iniciar a aplicação, acesse:

```
[text]
http://localhost:8080/swagger-ui/index.html
```

Também é possível acessar a especificação OpenAPI em formato JSON:

```
[text]
http://localhost:8080/v3/api-docs
```

A documentação Swagger permite visualizar os endpoints disponíveis, contratos de entrada, respostas esperadas e realizar testes diretamente pelo navegador.

Collections Postman

O projeto possui uma collection Postman para apoiar os testes manuais dos endpoints da API.

```
[text]
postman/Restaurant-API-Local.postman_collection.json
```

Como importar

1. Abra o Postman.
2. Clique em Import.
3. Selecione o arquivo `Restaurant-API-Local.postman_collection.json`.
4. Configure a variável `baseUrl`, se necessário, com o valor:

```
[text]
http://localhost:8080
```

A collection contempla cenários de sucesso e erro para autenticação, cadastro, consulta, atualização, troca de senha e exclusão de usuários.

Configuração e Execução

Pré-requisitos

Para execução local sem Docker, é necessário possuir:

- ? Java 17 instalado;
- ? Maven instalado ou uso do Maven Wrapper do projeto;
- ? PostgreSQL instalado e em execução;
- ? banco de dados `restaurant_db` criado.

Configuração padrão

O arquivo `src/main/resources/application.properties` utiliza variáveis de ambiente com valores padrão:

```
[properties]
spring.application.name=restaurant-management-api
server.port=8080

spring.datasource.url=${SPRING_DATASOURCE_URL:jdbc:postgresql://localhost:5432/restaurant_db}
spring.datasource.username=${SPRING_DATASOURCE_USERNAME:postgres}
spring.datasource.password=${SPRING_DATASOURCE_PASSWORD:postgres}

spring.sql.init.mode=${SPRING_SQL_INIT_MODE:always}
```

Executar localmente com Maven

No diretório raiz do projeto, execute:

```
[bash]
mvn spring-boot:run
```

Ou, utilizando o Maven Wrapper no Windows:

```
[bash]
mvnw.cmd spring-boot:run
```

No Linux/macOS:

```
[bash]
./mvnw spring-boot:run
```

Verificar se a aplicação subiu

```
[bash]
curl http://localhost:8080/health
```

Resposta esperada:

```
[text]
API is running
```

Ordem recomendada para avaliação manual

1. Subir o ambiente com Docker Compose.
2. Acessar `/health` para confirmar que a API está ativa.

3. Importar a collection Postman.
4. Criar um usuário cliente.
5. Criar um usuário dono de restaurante.
6. Listar usuários com GET /users.
7. Buscar usuário por ID.
8. Validar login com /auth/login.
9. Atualizar dados cadastrais.
10. Trocar senha.
11. Excluir usuário.

Docker e Docker Compose

A aplicação está preparada para execução containerizada, utilizando um container para o backend Java e outro para o PostgreSQL.

Serviços configurados

Serviço	Imagem/Base	Porta	Descrição
postgres	postgres:16-alpine	5432	Banco de dados relacional da aplicação.
app	Build local via Dockerfile	8080	API Spring Boot.

Executar com Docker Compose

No diretório raiz do projeto, execute:

```
[bash]  
docker compose up --build
```

Executar em segundo plano

```
[bash]  
docker compose up --build -d
```

Parar os containers

```
[bash]  
docker compose down
```

Parar e remover volumes do banco

Atenção: este comando remove os dados persistidos no volume do PostgreSQL.

```
[bash]  
docker compose down -v
```

Variáveis de ambiente suportadas

Variável	Valor padrão	Descrição
DB_NAME	restaurant_db	Nome do banco de dados criado no PostgreSQL.
DB_USER	postgres	Usuário do banco.
DB_PASSWORD	postgres	Senha do banco.
SPRING_DATASOURCE_URL	jdbc:postgresql://localhost:5432/restaurant_db	URL de conexão da aplicação.
SPRING_DATASOURCE_USERNAME	postgres	Usuário utilizado pela aplicação.
SPRING_DATASOURCE_PASSWORD	postgres	Senha utilizada pela aplicação.
SPRING_SQL_INIT_MODE	always	Controla a execução do schema.sql na inicialização.

Build, Testes e Validação

Compilar o projeto

No Windows:

```
[bash]
mvnw.cmd -DskipTests compile
```

No Linux/macOS:

```
[bash]
./mvnw -DskipTests compile
```

Executar testes automatizados

```
[bash]
mvnw.cmd test
```

OU:

```
[bash]
./mvnw test
```

Validação manual com Postman

A validação funcional recomendada para a Fase 01 deve utilizar a collection Postman disponível em:

```
[text]
postman/Restaurant-API-Local.postman_collection.json
```

Essa collection permite testar os fluxos exigidos no PDF: cadastro, consulta, alteração, exclusão, troca de senha e validação de login.

CrITÉRIOS de aceite da Fase 01

CrITÉrio do PDF	Como validar
Endpoints funcionam conforme descrito	Executar a collection Postman e conferir códigos HTTP/respostas.
Código organizado e documentado	Verificar camadas controller, service, repository, dto, entity, mapper e exception.
Documentação de arquitetura, endpoints e execução	Consultar este README e a documentação PDF gerada.

Critério do PDF	Como validar
Docker Compose configurado	Executar <code>docker compose up --build</code> .
Banco relacional integrado	Conferir conexão PostgreSQL e criação da tabela <code>users</code> .
Repositório aberto	Acessar o link GitHub informado nesta documentação.

Estrutura do Projeto

```
[text]
restaurant-management-api
├── Dockerfile
├── docker-compose.yml
├── mvnw
├── mvnw.cmd
├── pom.xml
├── README.md
├── postman
├── Restaurant-API-Local.postman_collection.json
├── src
│   ├── main
│   │   ├── java
│   │   │   ├── com
│   │   │   │   ├── restaurantsystem
│   │   │   │   │   ├── restaurantmanagementapi
│   │   │   │   │   │   ├── Config
│   │   │   │   │   │   │   ├── OpenApiConfig.java
│   │   │   │   │   │   │   ├── RestaurantManagementApiApplication.java
│   │   │   │   │   │   ├── controller
│   │   │   │   │   │   │   ├── AuthController.java
│   │   │   │   │   │   │   ├── HealthController.java
│   │   │   │   │   │   │   ├── UserController.java
│   │   │   │   │   │   ├── dto
│   │   │   │   │   │   │   ├── request
│   │   │   │   │   │   │   │   ├── AddressRequest.java
│   │   │   │   │   │   │   │   ├── LoginRequest.java
│   │   │   │   │   │   │   │   ├── PasswordUpdateRequest.java
│   │   │   │   │   │   │   │   ├── UserCreateRequest.java
│   │   │   │   │   │   │   │   ├── UserUpdateRequest.java
│   │   │   │   │   │   │   ├── response
│   │   │   │   │   │   │   │   ├── AddressResponse.java
│   │   │   │   │   │   │   │   ├── UserResponse.java
│   │   │   │   │   │   ├── entity
│   │   │   │   │   │   │   ├── Address.java
│   │   │   │   │   │   │   ├── User.java
│   │   │   │   │   │   ├── enums
│   │   │   │   │   │   │   ├── Role.java
│   │   │   │   │   │   ├── exception
│   │   │   │   │   │   │   ├── BusinessException.java
│   │   │   │   │   │   │   ├── ErrorResponse.java
│   │   │   │   │   │   │   ├── GlobalExceptionHandler.java
│   │   │   │   │   │   │   ├── UserNotFoundException.java
│   │   │   │   │   │   ├── mapper
│   │   │   │   │   │   │   ├── UserMapper.java
│   │   │   │   │   │   ├── repository
│   │   │   │   │   │   │   ├── UserRepository.java
│   │   │   │   │   │   ├── service
│   │   │   │   │   │   │   ├── UserService.java
│   │   │   │   │   │   │   │   ├── impl
│   │   │   │   │   │   │   │   ├── UserServiceImpl.java
│   │   │   │   │   ├── resources
│   │   │   │   │   │   ├── application.properties
│   │   │   │   │   │   ├── schema.sql
│   │   │   │   │   ├── test
│   │   │   │   │   │   ├── java
│   │   │   │   │   │   │   ├── com
│   │   │   │   │   │   │   │   ├── restaurantsystem
│   │   │   │   │   │   │   │   ├── restaurantmanagementapi
│   │   │   │   │   │   │   │   │   ├── RestaurantManagementApiApplicationTests.java
```

Aderência aos Requisitos da Fase 01

Requisito solicitado	Implementação no projeto	Status
Backend em Spring Boot	Projeto estruturado com Spring Boot, Spring Web MVC e Spring JDBC.	Atendido
Gerenciamento de usuários	Endpoints para cadastro, listagem, busca, atualização e exclusão.	Atendido
Dois tipos de usuário	Enum Role com CLIENT e RESTAURANT_OWNER.	Atendido
Cadastro com nome, e-mail, login, senha, data de alteração e endereço	Entidade User, objeto Address e DTOs de entrada/saída.	Atendido
Alteração de dados do usuário	Endpoint PUT /users/{id}.	Atendido
Troca de senha	Endpoint PATCH /users/{id}/password.	Atendido
Validação de login	Endpoint POST /auth/login.	Atendido
Banco relacional	PostgreSQL configurado no application.properties e no docker-compose.yml.	Atendido
Docker Compose	Arquivo docker-compose.yml com serviços app e postgres.	Atendido
Código organizado	Organização por camadas: controller, service, repository, dto, mapper, entity e exception.	Atendido
Documentação dos endpoints	Swagger/OpenAPI e seção de endpoints neste README.	Atendido
Collection para testes	Arquivo Postman disponível em postman/Restaurant-API-Local.postman_collection.json.	Atendido
Repositório de código	Projeto publicado em https://github.com/Andrefelix2810/restaurant-management-api .	Atendido

Decisões Técnicas

Separação entre DTO e entidade

O projeto utiliza DTOs para entrada e saída de dados, evitando expor diretamente os objetos de domínio nos contratos da API. Essa abordagem facilita a manutenção, melhora o controle sobre os dados trafegados e reduz o acoplamento entre a camada externa e o modelo de persistência.

Uso de camada de serviço

As regras de negócio foram centralizadas em UserServiceImpl, mantendo os controllers mais simples e focados apenas na comunicação HTTP. Essa separação favorece testes, reuso e evolução da aplicação.

Uso de mapper

A classe UserMapper concentra a conversão entre DTOs e entidades, evitando duplicação de código e mantendo a responsabilidade de transformação isolada.

Tratamento global de exceções

O uso de GlobalExceptionHandler permite padronizar as respostas de erro da API e melhorar a experiência de consumo dos endpoints.

Execução containerizada

O Docker Compose permite subir aplicação e banco de dados com um único comando, reduzindo problemas de configuração local e garantindo maior previsibilidade no ambiente de execução.

Possíveis Evoluções Futuras

Embora o projeto atenda ao escopo da Fase 01, algumas melhorias podem ser consideradas para fases posteriores:

- ? implementação de autenticação com JWT;
- ? criptografia de senha com BCrypt;
- ? inclusão de Spring Security;
- ? criação das entidades de restaurante, cardápio, pedido e avaliação;
- ? implementação de testes unitários e de integração mais abrangentes;
- ? inclusão de migrations com Flyway ou Liquibase;
- ? paginação e filtros na listagem de usuários;
- ? uso de perfis de ambiente, como dev, test e prod;
- ? implantação em ambiente cloud;
- ? melhoria da collection Postman com variáveis dinâmicas e scripts de teste.

Conclusão

A Restaurant Management API entrega uma base sólida para a primeira fase do sistema de gestão compartilhado de restaurantes. O projeto contempla os principais requisitos acadêmicos solicitados, apresentando uma API REST funcional, estruturada em camadas, integrada a banco relacional, documentada, testável via Postman e executável por Docker Compose.

A solução foi construída com foco em organização, clareza arquitetural e possibilidade de evolução, permitindo que novas funcionalidades do domínio de restaurantes sejam incorporadas nas próximas fases do projeto sem comprometer a estrutura atual.